

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naïve Bayes and KNN ₁
Student Name	Isha Venkataraman
Register Number	3122247001023
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	27/07/26

1 Objective

The goal of this experiment is to build a spam classifier for emails using two different families of machine learning algorithms, Naive Bayes and K-Nearest Neighbours, and to compare the performance of each. We test three variations of Naive Bayes, Gaussian, Multinomial and Bernoulli, since they make somewhat different assumptions about how the input features are distributed. For KNN, the experiment investigates the effect of the choice of k on the performance, compares two methods for the search of neighbours (KDTree and BallTree), and tunes the model with GridSearchCV and RandomizedSearchCV. Every model is judged using the same set of metrics — accuracy, precision, recall, F1-score and ROC-AUC — and 5-fold cross validation is used to check that the results are not just a fluke of one particular train-test split.

2 Problem Statement

Given a set of numeric features describing the email (mostly how often certain words or characters appear in it, plus a few features about runs of capital letters), the task is to predict if the email is spam or ham. This is a binary classification problem where the input for each email is 57 numeric values and one label which is 1 for spam and 0 for ham. The idea is not just to get one model to work, but to compare multiple algorithms on the same data and see which assumptions hold and which do not.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	Kaggle dataset hosted as <code>somesh24/spambase</code>
Number of Samples	4601 originally, 4210 after 391 duplicate rows were removed
Number of Features	57 numeric features (48 word frequency features, 6 character frequency features, 3 capital-run-length features)
Number of Classes	2 (spam = 1, ham = 0)
Missing Values	None found in either the original or deduplicated data
Train-Test Split	80/20, stratified on the class label (3368 training rows, 842 test rows)

4 Exploratory Data Analysis

Class Distribution

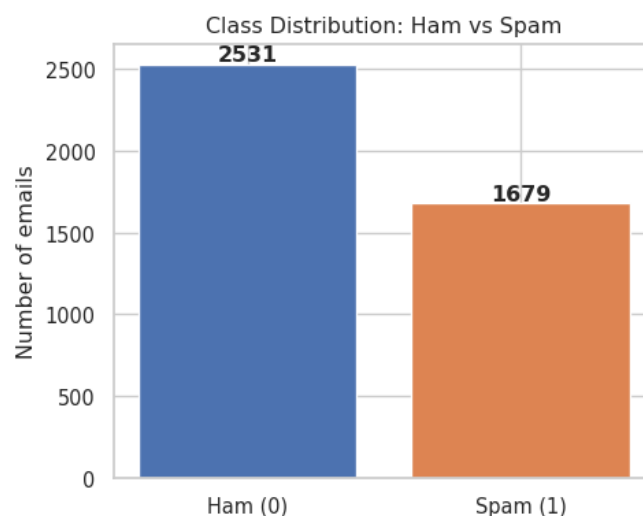


Figure 1: Distribution of ham vs spam emails after removing duplicate rows.

Inference: The dataset is slightly imbalanced with about 60% ham and 40% spam (2531 ham emails versus 1679 spam emails after duplicate removal). This is not bad enough to make accuracy meaningless on its own, but it is sufficiently bad that a model that just guesses ham every time would already score around 60% accuracy without learning anything useful, which is exactly why precision, recall, F1 and ROC-AUC are also reported later. A stratified split was used for the train-test division and for cross validation to keep the ratio of both classes roughly the same everywhere.

Correlation Matrix

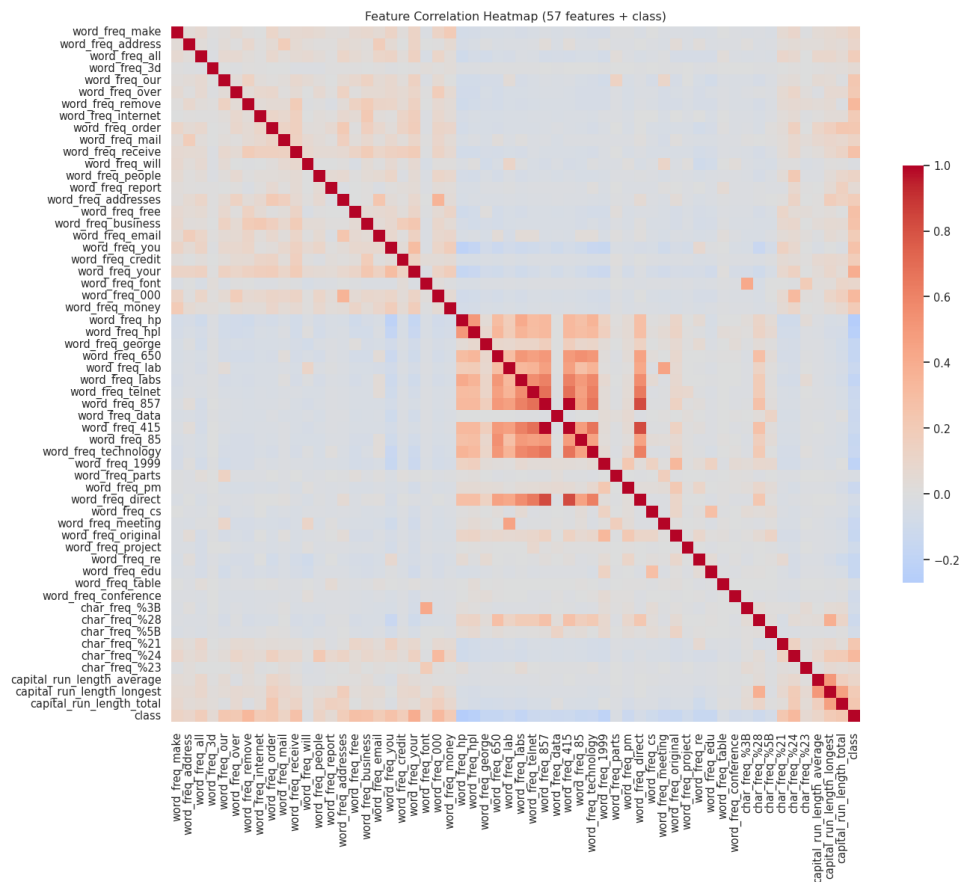


Figure 2: Correlation heatmap across all 57 features and the class label.

Inference: Most of the 57 features do not correlate strongly with each other, which makes sense because each one is measuring how often a different word or character appears, and different emails use different vocabulary. The features that do stand out as being more correlated with the class label are the ones associated with promotional or commercial language, for example `word_freq_your`, `word_freq_remove`, `word_freq_free`, `word_freq_000` and `char_freq_.`. A small cluster of features (things like `word_freq_650`, `word_freq_857`, `word_freq_415`, which look like area codes from whatever mailing list this dataset was originally built from) are strongly correlated with each other, which is worth remembering later since Gaussian Naive Bayes assumes conditional independence between features.

Feature Distribution

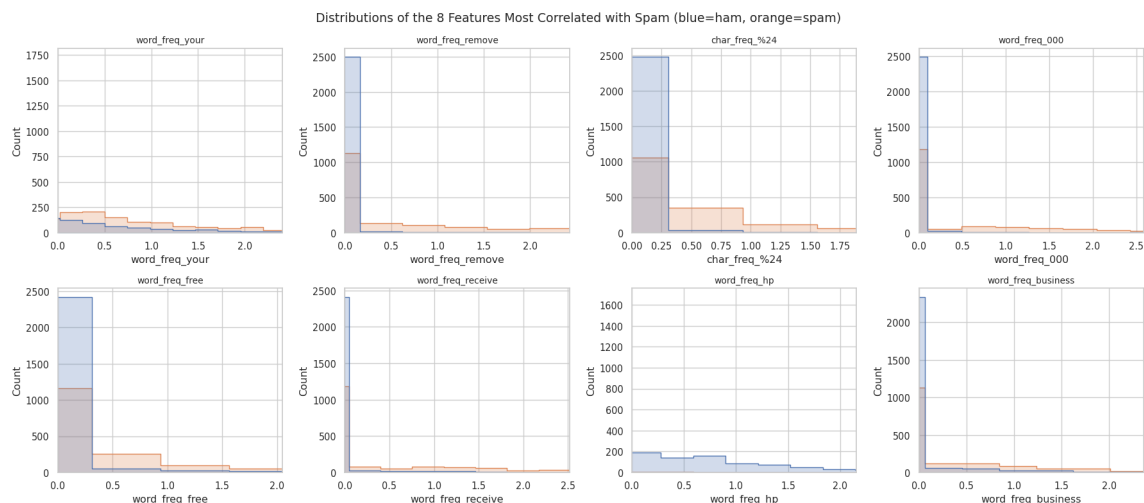


Figure 3: Histograms of the eight features most correlated with the class label, split by ham and spam.

Inference: All of these top features are heavily skewed toward zero, because most emails simply do not have a given word at all, and only a minority of emails use a word often. This is quite important for the modelling choices later, as Gaussian Naive Bayes assumes something close to a bell-shaped distribution for each feature, which is clearly not the case here, while Bernoulli and Multinomial Naive Bayes, which look at presence or count of words, fit this kind of sparse data much better. For words like *free*, *your* and *remove*, the spam distribution (in orange) is clearly shifted to the right of the ham distribution, which is essentially the signal that the classifiers detect.

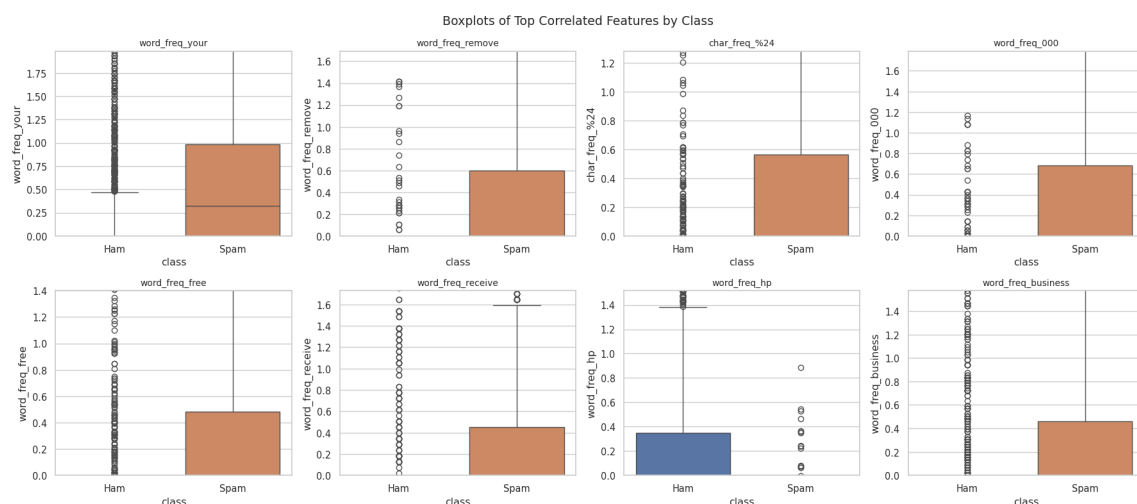


Figure 4: Boxplots of the same eight features, grouped by class.

Inference: In the boxplots the class separation is somewhat clearer. The spam median is higher than the ham median for almost all of these features. Spam also tends to have more extreme high-value outliers, for example in `char_freq_` and `word_freq_free`. This is similar to spam emails that repeat certain promotional words or symbols many

times in a single message. There are so many outliers across the board that all features were standardised before being fed into KNN, so that a few unusually extreme emails would not end up dominating the distance calculation.

Statistical Summary and Missing Value Analysis

Running `df.describe()` and `df.isnull().sum()` on the dataset confirmed there are zero missing values across all 57 feature columns and the class label, which is expected since these values are computed automatically from the raw email text rather than typed in by hand, leaving no real opportunity for blanks to appear. The summary statistics also confirmed that all frequency-based features are non-negative and mostly clustered near zero, consistent with what the histograms above show.

5 Data Preprocessing

Missing value treatment: There were no missing values in the dataset, thus no imputation was required. If any had been present, median imputation would have been the better choice given how skewed these features are; the mean would have been pulled around too much by the same outliers we saw in the boxplots.

Duplicate rows: There were 391 exact duplicate rows, which were removed before any modelling could be done. Leaving them in means that the same row might end up in both the train and test split later, meaning the model could “see” part of the test set during training and inflate reported scores.

Encoding: All the columns in Spambase are numeric, so there was no need for encoding of categorical data in this experiment.

Feature scaling: Two versions of the feature matrix were retained. One kept the frequency values as is (non-negative, unscaled), used for Multinomial NB, which expects count-like data, and Bernoulli NB, which only cares about presence or absence of a feature. The other version was normalised using `StandardScaler` (zero mean, unit variance) and used for Gaussian NB and for KNN. KNN uses Euclidean distance, and a feature like `capital_run_length_total`, which can be in the thousands, would otherwise dominate features that are only between 0 and 1.

Train-test split: An 80/20 split was used, stratified on the class label so that both the train and test sets keep roughly the same 60/40 ham-spam ratio as the full dataset. The same row indices were used to build both the raw and scaled versions of the training and test sets, so that all four models are compared on exactly the same emails.

6 Machine Learning Algorithm

Naive Bayes

Naive Bayes is a probabilistic classifier built on Bayes’ theorem, with the (fairly strong) assumption that features are conditionally independent given the class:

$$P(C | X) = \frac{P(X | C) P(C)}{P(X)} \quad (1)$$

It is the independence assumption that lets us decompose $P(X | C)$ as a product of per-feature likelihoods, which is what makes Naive Bayes so cheap to train even on

data with many features. The three variants used here differ only in the way they model $P(x_i | C)$ for each feature:

- Gaussian NB assumes that features are normally distributed within each class.
- Multinomial NB treats each feature as a count (how many times something occurs), good for frequency-style data.
- Bernoulli NB treats each feature as a binary indicator for presence/absence.

Advantages: It is very fast to train and predict, works quite well even with fairly little data and is not too badly affected by irrelevant features.

Limitations: The independence assumption rarely holds in real data (as seen with the correlated features in this dataset), and the variant choice must match the data's shape or performance suffers, as seen with Gaussian NB here.

K-Nearest Neighbours (KNN)

KNN is a lazy learner, meaning it does not really “train” a model at all — it just stores the training data. At prediction time, it finds the k closest training points to a new sample (using a distance metric such as Euclidean or Manhattan distance) and assigns the majority class among those neighbours:

$$\hat{y} = \text{mode}\{y_i : x_i \in N_k(x)\} \quad (2)$$

where $N_k(x)$ is the set of the k nearest neighbours of x . Two structures, KDTree and BallTree, are commonly used to make the neighbour search faster than brute force; though both return exactly the same neighbours as brute force, they just organise the search differently.

Advantages: The classifier does not assume anything about the shape of the data, can model non-linear decision boundaries and is easy to understand and implement.

Limitations: Prediction is slow on large datasets, as it has to look up (or index into) all the training data every time; it is sensitive to the scale of features (hence the need to standardise before using it), and performance depends a lot on choosing a good value of k .

7 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook
Operating System	Linux
Hardware	Standard CPU, no GPU acceleration used

8 Hyperparameter Selection

Parameter	Value
-----------	-------

KNN search space (<code>n_neighbors</code>)	1, 3, 5, 7, 9, 11, 13, 15
KNN search space (<code>weights</code>)	uniform, distance
KNN search space (<code>algorithm</code>)	kd_tree, ball_tree
KNN search space (<code>metric</code>)	euclidean, manhattan
GridSearchCV best <code>n_neighbors</code>	7
GridSearchCV best <code>weights</code>	distance
GridSearchCV best <code>metric</code>	manhattan
GridSearchCV best <code>algorithm</code>	kd_tree
RandomizedSearchCV <code>n_iter</code>	20
RandomizedSearchCV best <code>n_neighbors</code>	9
RandomizedSearchCV best <code>metric</code>	manhattan
Cross-validation strategy	Stratified 5-Fold, shuffled, <code>random_state</code> = 42

GridSearchCV explored all $8 \times 2 \times 2 \times 2 = 64$ combinations with 5 folds (320 fits in total) in ~ 42 seconds, achieving a mean CV accuracy of 0.9169. RandomizedSearchCV only sampled 20 combinations (100 fits) and took just over 12.5 seconds to run, with a mean CV accuracy of 0.9148, close enough to the grid search result that the much shorter runtime seems worth it for the very small drop in accuracy.

9 Results

Below are the results for the best overall model, the tuned KNN (`n_neighbors` = 7, `weights` = *distance*, `metric` = *manhattan*), on the held-out 20% test set.

Metric	Value
Accuracy	0.9287
Precision	0.9539
Recall	0.8631
F1-score	0.9063
ROC-AUC	0.9705

For reference, Table 2 below compares the three Naive Bayes variants against each other on the same test set.

Table 3: Naive Bayes variant comparison on the test set.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Gaussian NB	0.8290	0.7143	0.9524	0.8163	0.9471
Multinomial NB	0.7886	0.7616	0.6845	0.7210	0.8342
Bernoulli NB	0.8943	0.8777	0.8542	0.8658	0.9609

Bernoulli NB was the strongest of the three Naive Bayes variants on this dataset, going by F1-score.

Table 4: KNN accuracy, precision, recall and F1 as k is varied (uniform weights, Euclidean distance).

k	Accuracy	Precision	Recall	F1
1	0.9050	0.8832	0.8780	0.8806
3	0.9026	0.8944	0.8571	0.8754
5	0.9109	0.9143	0.8571	0.8848
7	0.9086	0.9164	0.8482	0.8810
9	0.9026	0.9178	0.8304	0.8719
11	0.9014	0.9121	0.8333	0.8709

Accuracy peaks at $k = 5$ in this sweep before drifting back down, which is why $k = 5$ was used as the baseline KNN model before any tuning was applied.

Table 5: GridSearchCV vs RandomizedSearchCV for KNN hyperparameter tuning.

Parameter	GridSearchCV	RandomizedSearchCV
Best k	7	9
Metric	manhattan	manhattan
Weights	distance	distance
Algorithm	kd_tree	kd_tree
CV Accuracy	0.9169	0.9148
Execution Time (s)	58.955	16.312

RandomizedSearchCV lands on almost the same combination as the full grid search (same metric, weighting and algorithm, a slightly different k), and gets to within 0.002 of the grid search's CV accuracy while taking roughly a quarter of the time, which is a fairly convincing case for using it over a full grid whenever the search space is large.

Table 6: KDTree vs BallTree for neighbour search in KNN.

Metric	KDTree	BallTree
Accuracy	0.910926	0.910926
Training Time (s)	0.023627	0.013441
Prediction Time (s)	0.250861	0.220359

As expected, KDTree and BallTree give exactly the same accuracy, since both return exactly the same neighbours as brute force, they only differ in how the search is organised internally. BallTree was faster on both training and prediction here, likely because Spambase has 57 dimensions, and KDTree's efficiency advantage tends to fade as dimensionality grows, which is exactly the regime BallTree was designed to handle better.

Table 7: 5-fold cross-validation accuracy for the best Naive Bayes variant (Bernoulli NB) and the tuned KNN.

Fold	Naive Bayes (Bernoulli)	Best KNN
1	0.8895	0.9192
2	0.8931	0.9287
3	0.8931	0.9121
4	0.8800	0.9086
5	0.8824	0.9157
Average	0.8876	0.9169

Tuned KNN beats Bernoulli NB on every single fold, not just on average, which is a stronger result than if it had only won on the original train-test split by chance.

Table 8: Theoretical time complexity, where n is the number of training rows and d is the number of features.

Algorithm	Training	Prediction
Naive Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)_{avg}$
BallTree	$O(n \log n)$	$O(\log n)_{avg}$

Table 9: Experimental training and prediction time for each model.

Algorithm	Training (s)	Prediction (s)
Gaussian NB	0.010432	0.005261
Multinomial NB	0.004088	0.001458
Bernoulli NB	0.005888	0.002350
Best KNN	0.022948	0.286109

The experimental numbers line up with the theoretical expectation above: all three Naive Bayes variants train and predict in a fraction of a second, while the tuned KNN is comparably fast to train (it just stores the data) but far slower to predict, since every test point has to be compared against the whole stored training set at inference time.

10 Result Visualization

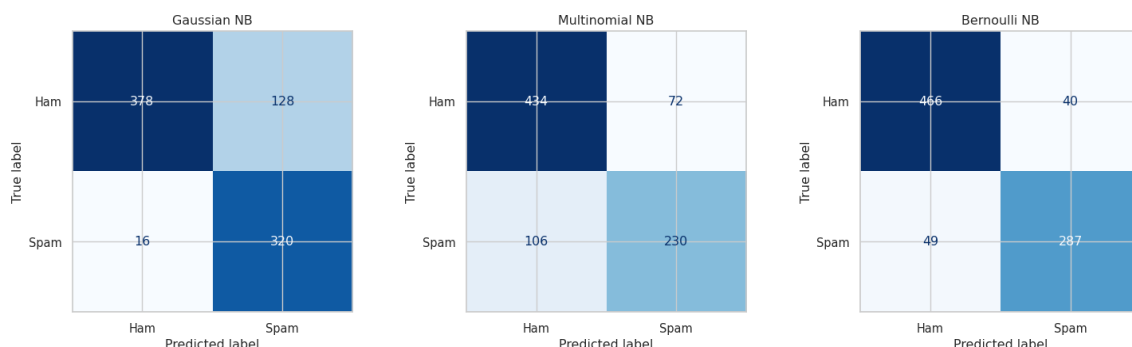


Figure 5: Confusion matrices for Gaussian, Multinomial and Bernoulli Naive Bayes on the test set.

Inference: Gaussian NB has the most off-diagonal entries in the confusion matrix of the three, and its errors are mostly false positives, that is, real ham emails are incorrectly classified as spam. That is a pretty expensive error for a real email filter, since it means you might block real mail. Overall, Bernoulli and Multinomial NB are less error-prone and their errors are more evenly spread between the two classes. This is shown in their improved F1 scores in the table above.

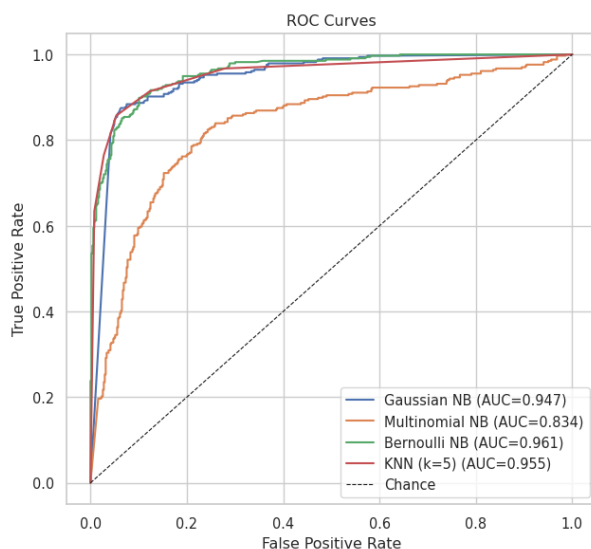


Figure 6: ROC curves for all four baseline models (three NB variants plus KNN with $k = 5$).

Inference: All four models are well above the diagonal “chance” line, so all four have learnt something real from the data. KNN and Bernoulli/Multinomial NB are closer to the top-left corner than Gaussian NB, which means that for almost any decision threshold they have a better true positive rate for the same false positive rate. Gaussian NB has a lower AUC than Bernoulli NB because the distributional assumption of Gaussian NB does not fit the actual shape of the data that was seen earlier in the EDA section.

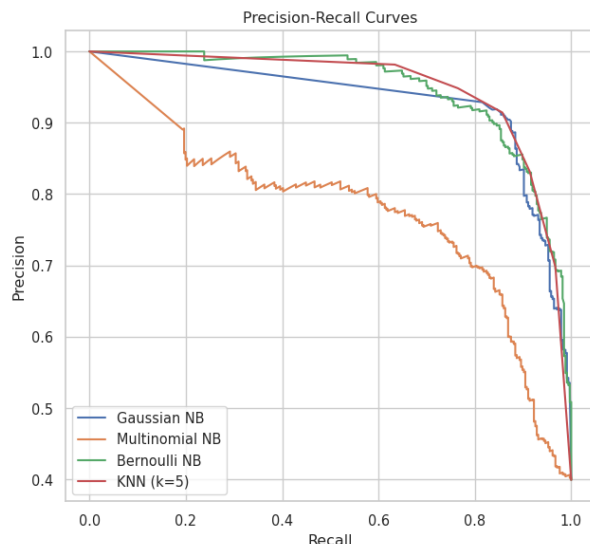


Figure 7: Precision-Recall curves for all four baseline models.

Inference: When there is class imbalance, the Precision-Recall curve is more important than the ROC curve because it shows how well the positive (spam) class is being handled. Models that have curves staying near precision = 1 as recall increases are catching more spam without misclassifying too much ham along the way. At high recall, the Gaussian NB curve drops off the fastest, meaning it is sacrificing a lot of precision to catch the last few spam emails. This again goes back to its normal distribution assumption not holding for this data.

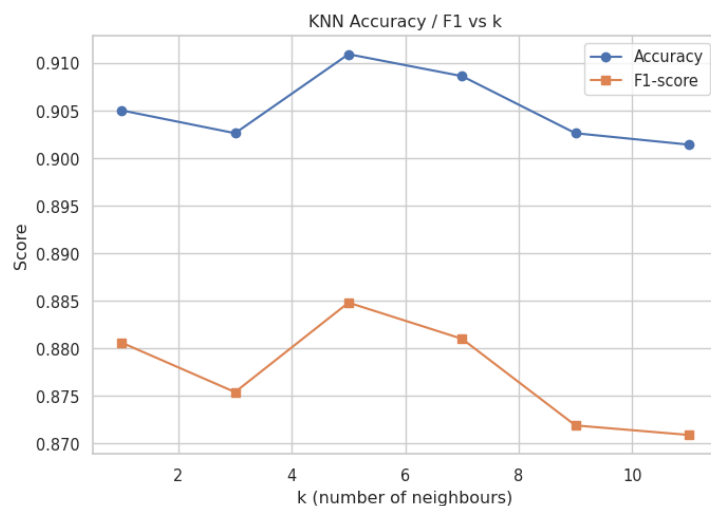


Figure 8: KNN accuracy and F1-score as k is varied from 1 to 11.

Inference: At $k = 1$, the model is literally memorising the training data, since a prediction is based only on one neighbour and is thus sensitive to noise or mislabelled points. As k increases, predictions are averaged over more neighbours, which smooths the decision boundary and helps generalisation for a while. Push k too far, though, and the neighbourhood becomes large enough to start pulling in points from the other class, dragging predictions towards the majority class everywhere. This is the underfitting side

of the curve. For this dataset, accuracy peaks somewhere in the middle of the range tested, not at the extremes.

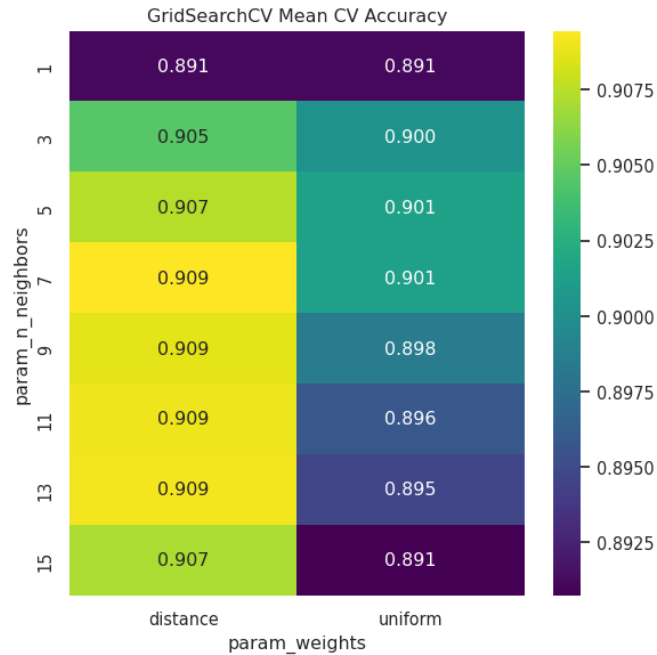


Figure 9: Mean cross-validation accuracy for different combinations of k and weighting scheme from GridSearchCV.

Inference: This heatmap characterises the interaction between k and the weighting scheme. Distance weighting (where closer neighbours get a stronger vote) tends to help more when k is small, since a single very close neighbour is often more reliable than several farther ones. Looking down a column shows how sensitive the accuracy is to the exact value of k for a given weighting scheme. A flatter column means the model is fairly robust to the choice of k , while a column with a sharp peak means k needs to be chosen carefully.

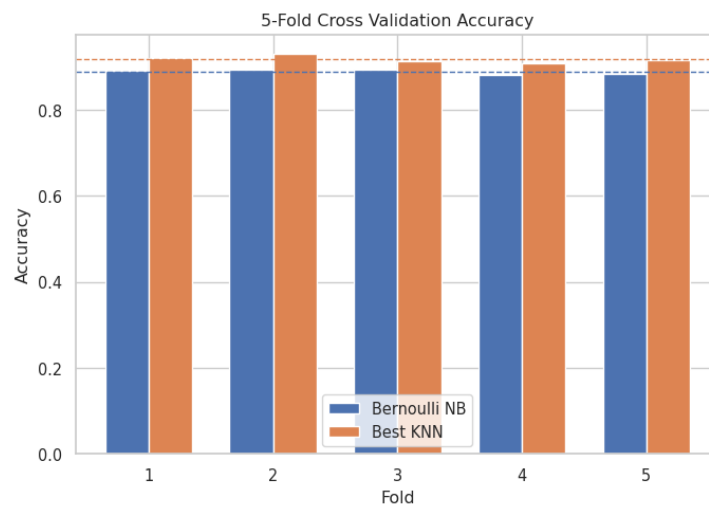


Figure 10: 5-fold cross-validation accuracy for the best Naive Bayes variant (Bernoulli NB) and the tuned KNN.

Inference: The accuracy does not vary much from fold to fold for either model (Bernoulli NB averaged 0.8876, tuned KNN averaged 0.9169), which suggests that the earlier single train-test split result was not just a lucky split. It is consistent with the ability of KNN to leverage correlated feature interactions that are ignored under the independence assumption of Naive Bayes, but at a real cost in prediction speed, discussed next. Tuned KNN beats Bernoulli NB consistently over all five folds.

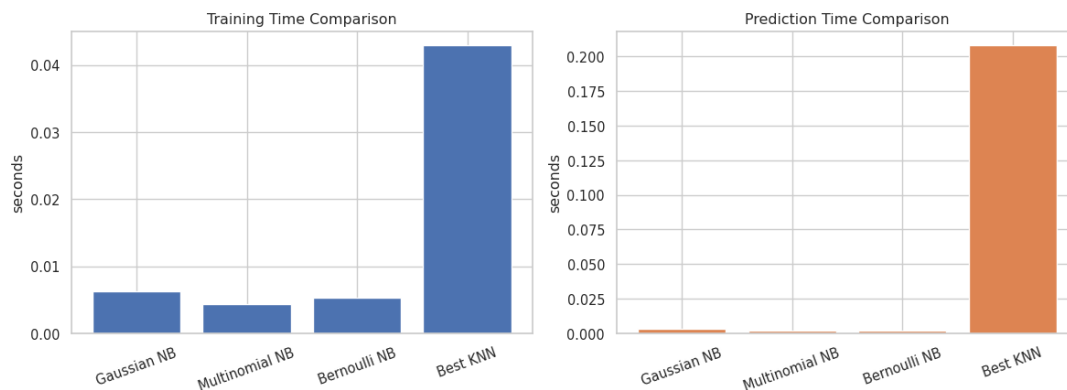


Figure 11: Training and prediction time for the three Naive Bayes variants and the tuned KNN.

Inference: All three variants of Naive Bayes train and predict in near-instant time, consistent with the theoretical $O(nd)$ training and $O(d)$ prediction complexity, since Naive Bayes only requires a single pass over the data to compute per-class, per-feature statistics. Of the four, KNN is the fastest in its “training” phase, as it is essentially just storing data and not learning anything, but it is noticeably the slowest in its prediction phase, since every test point needs to be compared against the stored training points during inference. This experimentally confirms the theoretical expectation that KNN pushes almost all of its computational cost to prediction time rather than training time.

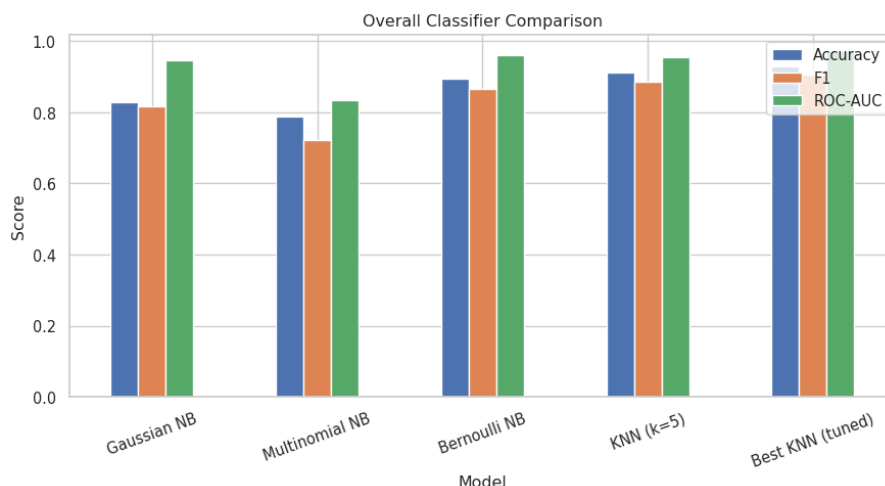


Figure 12: Overall comparison of accuracy, F1-score and ROC-AUC across all five models tested.

Inference: This chart brings it all together in one place. The tuned KNN outperforms all Naive Bayes variants on all three metrics presented. Comparing it to the untuned KNN

($k = 5$) shows that hyperparameter tuning delivered a real, albeit small, improvement in performance. Bernoulli NB is still a strong, much cheaper second choice — this will matter if training/prediction speed becomes a real constraint at a larger scale.

11 Discussion

When looking back at all of this, the biggest thing that stood out was just how much the choice of Naive Bayes variant mattered. Gaussian NB, despite being the “default” choice that most people try first, was actually the worst of the three here, simply because the underlying features are nowhere near normally distributed — they are skewed and full of zeros. Bernoulli NB just checks for the presence of a word or not, and did pretty well. This could be because, for spam detection, the presence of a trigger word like “free” or “remove” is often enough information on its own, without needing to know the exact count.

KNN needed more tuning to get right, as it depends on scaling, choice of k and distance metric all at once, but once tuned it edged out every Naive Bayes variant on precision, F1 and ROC-AUC. The catch is prediction speed: KNN was by far the slowest model to run predictions with, by a long shot compared to any of the Naive Bayes variants. This lines up exactly with what the theoretical time complexity would predict. So there is a real trade-off here, not just “one model wins outright”: KNN gives you better accuracy, Naive Bayes gives you much faster predictions, and which one actually makes sense depends on the scale you are deploying at.

One limitation of this experiment is that we only used one train-test split (plus 5-fold CV on top of it) to compare things. If we ran this a few more times with different random seeds, we would get a better sense of how much these numbers can vary. It would also be worth trying an ensemble of KNN and Bernoulli NB together, as their errors do not look identical, so combining them could plausibly do better than either alone. Another improvement to try would be feature selection, as the correlation heatmap showed a few features that are almost redundant with each other, and dropping some of them might speed up KNN prediction without losing much accuracy.

12 Conclusion

This experiment was meant to compare Naive Bayes and KNN for spam classification on the Spambase dataset, and it turned out to be a decent illustration of why assumptions about the algorithm are just as important as the algorithm choice itself. Bernoulli NB performed best among the Naive Bayes variants, as its presence/absence assumption fits this type of skewed, zero-heavy word-frequency data much better than the normality assumption of Gaussian NB. With appropriate tuning using GridSearchCV, the KNN model was the best-performing model overall on accuracy, precision, F1 and ROC-AUC, but at a significantly higher prediction-time cost, which would be more relevant at scale. On the whole, for a moderate-sized dataset like this one where accuracy is the main priority, tuned KNN looks like the better choice, while Naive Bayes remains attractive wherever prediction speed or simplicity matters more.

13 References

1. Spambase Dataset from Kaggle: <https://www.kaggle.com/datasets/somesh24/spambase>
2. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
3. A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2019.
4. C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.